

MinIO on Oxide: Lab Guide

What you'll build	3
Prerequisites	3
Phase 0: Pre-flight	3
Phase 1: Networking	4
Phase 2: MinIO compute and storage	7
Sizing	8
Create instances via JSON body	8
Attach instances to the anti-affinity group	10
Phase 3: Load balancer compute and Floating IP	12
Load Balancer Infrastructure	12
Phase 4: OS preparation on each node	15
Phase 5: MinIO install and cluster formation	20
Name resolution	20
Generate strong root credentials	21
Push the finalized /etc/default/minio to all 4 nodes	21
Start the cluster	22
Validate with mc	23
Phase 6: HAProxy + watcher + FIP failover	24
Hosts entries on LBs	24
Self-signed TLS certificate	24
HAProxy config	25
Smoke test via FIP	26
Failover mechanism (oxide CLI + credentials + watcher)	26
Install Oxide CLI on both LBs	26
Copy laptop's credentials to root on each LB	26
Failover drill	29
Architecture FAQ	30
Software choices	30
Why MinIO and not Ceph, Swift, or RustFS?	30
Why HAProxy instead of NGINX?	30
Why customer-built LB instead of an Oxide-native one?	30
Why systemd, not containers or Kubernetes?	30
Why Ubuntu 24.04 LTS?	30

Topology.....	30
Why 4 MinIO instances?.....	30
Why 4 drives per instance?.....	31
Why 2 LB instances?.....	31
Why a dedicated VPC (`minio-vpc`) instead of the default VPC?.....	31
In a distributed MinIO cluster, which node "owns" a bucket or object?.....	31
Storage choices.....	31
Why local disks for data, but distributed for the boot disk?.....	31
Why EC:4 (12 data + 4 parity) instead of mirroring or other parity counts?.....	32
Why 100 GiB data disks and 30 GiB boot disks?.....	32
Why block size 4096 on data disks but 512 on the boot disk?.....	32
Why is sled placement verification mandatory?.....	32
Why does the bootstrap script discover the boot disk dynamically instead of assuming nvme0n1?.....	32
Networking choices.....	32
Why a Floating IP and not DNS round-robin?.....	32
Why TLS terminated at the LB rather than end-to-end to MinIO?.....	33
Why ephemeral external IPs during the build, but planning to detach later?.....	33
Why doesn't the firewall allow port 9000 from the internet, even though clients need to reach MinIO?.....	33
Why is the Floating IP sticky after failover instead of bouncing back to the primary?.....	33

This is a self-contained, phase-by-phase build guide. Read it top to bottom, run the commands, and you'll end with a working S3 endpoint backed by a 4-node MinIO cluster on a single Oxide rack.

For the design rationale behind these choices, see ARCHITECTURE doc in the project root. This guide is the execution layer.

What you'll build

- 4 Oxide instances running MinIO in distributed mode (16 local disks total, EC:4)
- 2 small Oxide instances running HAProxy as an active/standby LB pair
- 1 Floating IP fronting the LB, exposing a single S3 endpoint over TLS
- All inside one Oxide project, one VPC

Prerequisites

- Oxide silo with sufficient rights to create projects, instances, disks, VPCs, firewall rules, and Floating IPs
- *oxide* CLI installed on your workstation
- At least one IP pool linked to your silo
- About 90 minutes for the full build, not counting OS package downloads

Phase 0: Pre-flight

Goal: confirm CLI is pointed at the right silo, create the project that holds everything, verify IP pool linkage.

1. Authenticate the CLI:

Shell

```
oxide auth login --host https://<your-silo>.sys.<rack>.oxide-preview.com
```

2. Confirm which silo the CLI is talking to:

Shell

```
oxide auth status
```

3. If you have more than one auth profile, pin to the silo you want for this build:

Shell

```
export OXIDE_HOST=https://<your-silo>.sys.<rack>.oxide-preview.com
```

4. Create the project:

Shell

```
oxide project create --name minio-poc --description "MinIO POC infrastructure"
```

Or use the console: top-left, Projects, New project.

5. Verify an IP pool is linked to your silo:

Shell

```
oxide ip-pool list
```

You want at least one pool back, ideally with *is_default*: *true*. If empty, ask your fleet admin to link one before continuing.

💡 If the *oxide project list* says "not found" for a project you definitely created in the console, your CLI is authenticated against a different silo than the console session. Run *oxide auth status* to see every profile your CLI has. Use *OXIDE_HOST* to pin commands to the silo you want.

Verify setup with below commands:

- *oxide project list* returns *minio-poc*
- *oxide ip-pool list* returns at least one pool
- *oxide floating-ip list --project minio-poc* returns [] without error

Phase 1: Networking

Goal: dedicated VPC, dedicated subnet, and firewall rules ready for instance creation.

1. Create the MinIO VPC:

Shell

```
oxide vpc create \
  --project minio-poc \
  --name minio-vpc \
```

```
--description "MinIO service VPC" \
--dns-name minio
```

2. Delete the auto-created *default* subnet that Oxide adds inside every new VPC:

Shell

```
oxide vpc subnet delete --project minio-poc --vpc minio-vpc --subnet default
```

3. Create the application subnet:

Shell

```
oxide vpc subnet create \
  --project minio-poc \
  --vpc minio-vpc \
  --name minio-subnet \
  --description "Subnet for MinIO nodes and LB instances" \
  --ipv4-block 10.10.0.0/24
```

4. Inspect the firewall rules Oxide auto-created. You should see *allow-icmp*, *allow-internal-inbound*, and *allow-ssh*:

Shell

```
oxide vpc firewall-rules view --project minio-poc --vpc minio-vpc
```

5. Add *allow-https-inbound* so clients can reach the S3 endpoint on TCP 443 once the Floating IP is in place. Easiest path is the console (VPC, Firewall Rules, New rule). For CLI, save the JSON below as *firewall-rules.json*, then apply.

The firewall API replaces the entire rule set per call, so the file must contain all existing rules plus the new one:

JSON

```
{
  "rules": [
    {
      "name": "allow-icmp",
      "description": "allow inbound ICMP traffic from anywhere",
      "priority": 65534,
```

```

        "action": "allow",
        "direction": "inbound",
        "status": "enabled",
        "targets": [{ "type": "vpc", "value": "minio-vpc" }],
        "filters": { "protocols": [{ "type": "icmp" }] }
    },
    {
        "name": "allow-internal-inbound",
        "description": "allow inbound traffic to all instances within the VPC if
originated within the VPC",
        "priority": 65534,
        "action": "allow",
        "direction": "inbound",
        "status": "enabled",
        "targets": [{ "type": "vpc", "value": "minio-vpc" }],
        "filters": { "hosts": [{ "type": "vpc", "value": "minio-vpc" }] }
    },
    {
        "name": "allow-ssh",
        "description": "allow inbound TCP connections on port 22 from anywhere",
        "priority": 65534,
        "action": "allow",
        "direction": "inbound",
        "status": "enabled",
        "targets": [{ "type": "vpc", "value": "minio-vpc" }],
        "filters": { "ports": ["22"], "protocols": [{ "type": "tcp" }] }
    },
    {
        "name": "allow-https-inbound",
        "description": "Allow TCP 443 inbound for MinIO S3 endpoint via Floating
IP",
        "priority": 100,
        "action": "allow",
        "direction": "inbound",
        "status": "enabled",
        "targets": [{ "type": "vpc", "value": "minio-vpc" }],
        "filters": { "ports": ["443"], "protocols": [{ "type": "tcp" }] }
    }
]
}

```

Apply:

Shell

```
oxide vpc firewall-rules update \
--project minio-poc \
--vpc minio-vpc \
--json-body firewall-rules.json
```

💡 The firewall API does not have an "add one rule" endpoint. Every update replaces the entire rule set, so you must include existing rules in the payload.

Verify setup with below commands:

- `oxide vpc list --project minio-poc` returns default and minio-vpc.
- `oxide vpc subnet list --project minio-poc --vpc minio-vpc` returns one subnet: minio-subnet at 10.10.0.0/24.
- `oxide vpc firewall-rules view --project minio-poc --vpc minio-vpc` returns four rules including allow-https-inbound.

Phase 2: MinIO compute and storage

Goal: 4 MinIO instances spread across 4 distinct sleds, each with 1 boot disk and 4 local data disks, all in *minio-vpc/minio-subnet*, all SSH-reachable.

1. Get an Ubuntu 24.04 LTS image into the project. If the silo doesn't have one pre-loaded, download from *cloud-images.ubuntu.com*, convert from qcow2 to raw, upload via the console "Create image" wizard.

Shell

```
gemu-img convert -f qcow2 -O raw \
ubuntu-24.04-server-cloudimg-amd64.img \
ubuntu-24.04-server-cloudimg-amd64.raw
```

Shell

```
gemu-img info ubuntu-24.04-server-cloudimg-amd64.raw
```

Image upload settings: format *raw*, block size 512. Ubuntu cloud images have GPT laid out for 512-byte sectors; overriding to 4096 can cause silent boot failures.

2. Register your SSH public key with Oxide:

Shell

```
ssh-keygen -t ed25519 -C "your-workstation"
oxide current-user ssh-key create \
```

```
--name <your-key-name> \
--description "Workstation SSH public key" \
--public-key "$(cat ~/.ssh/id_ed25519.pub)"
```

3. Confirm at least one IP pool is linked with free capacity. Each instance with a default external IP consumes two pool addresses (ephemeral + SNAT). If the silo's default pool is short, link a second pool as fleet admin:

Shell

```
oxide system networking ip-pool silo link \
  --pool <pool-name> \
  --silo <silo-name> \
  --is-default false \
  --profile <fleet-admin-profile>
```

4. Create an anti-affinity group for the MinIO cluster. This forces the scheduler to place each MinIO instance on a different sled.

Shell

```
oxide instance anti-affinity create \
  --project minio-poc \
  --name minio-spread \
  --description "Force MinIO nodes onto different sleds" \
  --policy fail \
  --failure-domain sled
```

policy: fail means instance creation FAILS if the constraint can't be honored, which is what we want

Sizing

- MinIO instance count: 4
- vCPU per instance: 4
- RAM per instance: 16 GiB (17179869184 bytes)
- Boot disk per instance: 30 GiB, disk_backend: distributed
- Data disks per instance: 4 x 100 GiB, disk_backend: local, block size 4096
- Total raw storage: 1.6 TiB
- Anti-affinity group: minio-spread (policy fail, domain sled)

Create instances via JSON body

The clean path is `oxide api --method POST /v1/instances?project=<project> --input <file.json>`. Per-instance JSON template, then submit in a loop. Adjust `IMAGE_ID`, `SSH_KEY_NAME`, and `POOL_NAME`.

JSON

```
{
  "name": "minio-inst-N",
  "hostname": "minio-inst-N",
  "description": "MinIO node N of 4",
  "ncpus": 4,
  "memory": 17179869184,
  "start": true,
  "boot_disk": {
    "type": "create",
    "name": "minio-inst-N-os",
    "description": "Boot disk for minio-inst-N",
    "size": 32212254720,
    "disk_backend": {
      "type": "distributed",
      "disk_source": {
        "type": "image",
        "image_id": "<IMAGE_ID>"
      }
    }
  },
  "disks": [
    {"type": "create", "name": "minio-inst-N-dd1", "description": "Data disk 1", "size": 107374182400, "disk_backend": {"type": "local", "disk_source": {"type": "blank", "block_size": 4096}}},
    {"type": "create", "name": "minio-inst-N-dd2", "description": "Data disk 2", "size": 107374182400, "disk_backend": {"type": "local", "disk_source": {"type": "blank", "block_size": 4096}}},
    {"type": "create", "name": "minio-inst-N-dd3", "description": "Data disk 3", "size": 107374182400, "disk_backend": {"type": "local", "disk_source": {"type": "blank", "block_size": 4096}}},
    {"type": "create", "name": "minio-inst-N-dd4", "description": "Data disk 4", "size": 107374182400, "disk_backend": {"type": "local", "disk_source": {"type": "blank", "block_size": 4096}}}
  ],
  "network_interfaces": {
    "type": "create",
    "params": [
      {
        "name": "minio-inst-N-nic",
        "description": "Primary NIC in minio-vpc",

```

```

        "vpc_name": "minio-vpc",
        "subnet_name": "minio-subnet"
    }
]
},
"external_ips": [
    {"type": "ephemeral", "pool": "<POOL_NAME>"}
],
"ssh_public_keys": [ "<SSH_KEY_NAME>" ]
}

```

Submit the four in parallel (background each call to avoid HTTP timeout blocking), using below bash script.

```

Shell
for n in 1 2 3 4; do
    # Populate /tmp/minio-inst-{$n}.json from the template, substituting N.
    oxide api \
        --method POST "/v1/instances?project=minio-poc" \
        --input /tmp/minio-inst-{$n}.json &
done
wait

```

HTTP timeouts during create are normal; provisioning continues server-side. Poll `oxide instance list --project minio-poc` until all four show *run_state: running*.

Attach instances to the anti-affinity group

After each instance reaches *running*, stop the instances, add them to *minio-spread*, and then start the instances

```

Shell
for n in 1 2 3 4; do
    oxide instance anti-affinity member add \
        --project minio-poc \
        --anti-affinity-group minio-spread \
        --instance minio-inst-{$n}
done

```

Verify Instance state and placement:

Shell

```
oxide instance list --project minio-poc
oxide instance nic list --project minio-poc --instance minio-inst-1
```

vpc_id should match *minio-vpc*. Private IP should fall inside *10.10.0.0/24*.

Sled placement (must show 4 unique sleds for the 4 MinIO instances), using the below bash script:

Shell

```
#!/usr/bin/env bash
# sled-map.sh
echo "=== Sled placement for MinIO cluster ==="
for sled_id in $(oxide system hardware sled list --profile <fleet-admin-profile>
jq -r '.[].id'); do
    matches=$(oxide system hardware sled instance-list \
        --profile <fleet-admin-profile> \
        --sled-id "$sled_id" 2>/dev/null \
        jq -r '.[] | select(.name startswith("minio-inst-")) | .name' 2>/dev/null)
    if [ -n "$matches" ]; then
        echo ""
        echo "Sled $sled_id:"
        echo "$matches" sed 's/^/ - /'
    fi
done
```

With *minio-spread* active, this should report 4 distinct sleds, one MinIO per sled, every time.

SSH into one node and confirm storage:

Shell

```
ssh ubuntu@<external-ip>
lsblk          # 1 boot disk with partitions + 4 unformatted 100 GiB devices
nproc          # 4
free -h        # ~15 GiB
df -h /        # ~27 GiB free
```

Verify setup with below commands:

1. All four instances show *running* in *oxide instance list*.
2. Every NIC reports *vpc_id* matching *minio-vpc* and a private IP in *10.10.0.0/24*.
3. Sled-map shows 4 unique sleds.

4. SSH succeeds to every node; *lsblk* shows 1 boot + 4 data disks.
5. All four instances are members of the *minio-spread* anti-affinity group.

Phase 3: Load balancer compute and Floating IP

Goal: 2 LB instances on 2 different sleds + 1 Floating IP attached to lb-1 as the active endpoint.

Load Balancer Infrastructure

- LB instance count: 2
- vCPU per instance: 2
- RAM per instance: 4 GiB (4294967296 bytes)
- Boot disk per instance: 20 GiB, disk_backend: distributed
- Data disks: none
- External IPs: none (Floating IP handles ingress, ProxyJump handles SSH)
- Anti-affinity group: lb-spread (policy fail, domain sled)

1. Create the LB anti-affinity group:

Shell

```
oxide instance anti-affinity create \  
  --project minio-poc \  
  --name lb-spread \  
  --description "Force LB instances onto different sleds" \  
  --policy fail \  
  --failure-domain sled
```

2. Create lb-1 and lb-2 via JSON-body POST. Per-instance template:

JSON

```
{  
  "name": "lb-N",  
  "hostname": "lb-N",  
  "description": "HAProxy LB N of 2",  
  "ncpus": 2,  
  "memory": 4294967296,  
  "start": true,  
  "boot_disk": {  
    "type": "create",  
    "name": "lb-N-os",  
    "description": "Boot disk for lb-N",
```

```

    "size": 21474836480,
    "disk_backend": {
      "type": "distributed",
      "disk_source": {
        "type": "image",
        "image_id": "<IMAGE_ID>"
      }
    },
    "disks": [],
    "network_interfaces": {
      "type": "create",
      "params": [
        {
          "name": "lb-N-nic",
          "description": "Primary NIC in minio-vpc",
          "vpc_name": "minio-vpc",
          "subnet_name": "minio-subnet"
        }
      ]
    },
    "external_ips": [],
    "ssh_public_keys": ["<SSH_KEY_NAME>"]
  }
}

```

Submit both in parallel using bash script below:

Shell

```

for n in 1 2; do
  oxide api \
    --method POST "/v1/instances?project=minio-poc" \
    --input /tmp/lb-${n}.json &
done
wait

```

3. Add both LBs to *lb-spread* after they reach *running*:

Shell

```

for n in 1 2; do
  oxide instance anti-affinity member add \
    --project minio-poc \

```

```

--anti-affinity-group lb-spread \
--instance lb-{$n}
done

```

4. Verify sled placement (lb-1 and lb-2 must be on different sleds):

Shell

```

for sled_id in $(oxide system hardware sled list --profile <fleet-admin-profile>
jq -r '[][.id]'); do
    matches=$(oxide system hardware sled instance-list \
        --profile <fleet-admin-profile> \
        --sled-id "$sled_id" 2>/dev/null \
        jq -r '[][.name | select(.name | startswith("lb-")) | .name]' 2>/dev/null)
    if [ -n "$matches" ]; then
        echo "Sled $sled_id:"; echo "$matches" | sed 's/^/  - /'
    fi
done

```

5. Allocate the Floating IP and attach to lb-1:

Shell

```

oxide floating-ip create \
    --project minio-poc \
    --name minio-s3-endpoint \
    --description "S3 endpoint for MinIO cluster, attaches to active LB VM"

oxide floating-ip attach \
    --project minio-poc \
    --floating-ip minio-s3-endpoint \
    --kind instance \
    --parent lb-1

oxide floating-ip view --project minio-poc --floating-ip minio-s3-endpoint

```

💡 Omit the pool flag to draw from the silo's default pool (which is fine when the default is externally routable like Oxide's *public* pool). Specify `--ip-pool <name>` if you need a specific pool. The view output should show *instance_id* matching lb-1.

Verify reachability from your workstation (no VPN needed if the FIP is on a publicly-routable pool):

```
Shell
ping -c 3 <floating-ip>
```

allow-icmp lets it through. A response confirms the FIP is fully routed to lb-1's NIC.

SSH access during build

LBs have no external IPs. Reach them via ProxyJump through a MinIO instance that does:

```
None
Host lb-1
  HostName <lb-1 private IP>
  User ubuntu
  ProxyJump minio-inst-1

Host lb-2
  HostName <lb-2 private IP>
  User ubuntu
  ProxyJump minio-inst-1
```

lb-1 should become reachable now via the Floating IP directly.

Verify setup with below commands:

1. Both LB instances show `running`.
2. Both NICs reside in `minio-vpc`, private IPs in `10.10.0.0/24`.
3. Sled-map shows lb-1 and lb-2 on different sleds.
4. SSH succeeds via ProxyJump.
5. Both instances are members of `lb-spread`.
6. Floating IP attached to lb-1, reachable from a client on the same network as the FIP pool.

Phase 4: OS preparation on each node

Goal: each MinIO node has its data disks formatted and mounted, the MinIO binary installed under systemd, and a placeholder environment file. Each LB node has HAProxy and keepalived installed.

MinIO bootstrap script

Run this on every MinIO instance. Idempotent (safe to re-run).

Shell

```
#!/usr/bin/env bash
set -euo pipefail
echo "=== Bootstrapping MinIO node $(hostname) ==="

# Packages
sudo apt update
sudo apt upgrade -y
sudo apt install -y xfsprogs curl jq chrony
sudo systemctl enable --now chrony
```

Shell

```
# minio user
if ! id minio >/dev/null 2>&1; then
    sudo useradd --system --no-create-home --shell /sbin/nologin minio
fi
```

Shell

```
# Discover the boot device so we don't accidentally format it (NVMe device
# numbering is NOT stable across Oxide instances - see gotcha below).
boot_dev=$(findmnt -no SOURCE / sed 's/dev/; sp[0-9]*$')
echo "Boot device detected: $boot_dev"

# Find the four data disks (any nvme*n1 device that isn't the boot device).
mapfile -t data_devs < <(lsblk -dn -o NAME grep -E '^nvme[0-9]+n1$' grep -v
"^${boot_dev}$")

if [ ${#data_devs[@]} -ne 4 ]; then
    echo "ERROR: expected 4 data disks, found ${#data_devs[@]}"; exit 1
fi

# Format and mount each data disk
for i in "${!data_devs[@]}"; do
    slot=$((i + 1))
    dev="/dev/${data_devs[$i]}"
    mp="/mnt/disk${slot}"
    if ! sudo blkid "$dev" >/dev/null 2>&1; then
        sudo mkfs.xfs -f -L "disk${slot}" "$dev"
    fi
    sudo mkdir -p "$mp"
    uuid=$(sudo blkid -s UUID -o value "$dev")
```

```
if ! grep -q "$uuid" /etc/fstab; then
    echo "UUID=${uuid} ${mp} xfs defaults,noatime,inode64 0 0" | sudo tee -a
/etc/fstab
fi
done
sudo systemctl daemon-reload
sudo mount -a
```

Shell

```
# MinIO subdirectories
for slot in 1 2 3 4; do
    sudo mkdir -p "/mnt/disk${slot}/minio"
    sudo chown -R minio:minio "/mnt/disk${slot}"
done
```

Shell

```
# MinIO binary
if [ ! -x /usr/local/bin/minio ]; then
    sudo curl -L https://dl.min.io/server/minio/release/linux-amd64/minio -o
/usr/local/bin/minio
    sudo chmod +x /usr/local/bin/minio
fi
```

Shell

```
# Environment file (Phase 5 finalizes MINIO_VOLUMES + credentials)
sudo tee /etc/default/minio >/dev/null <<'EOF'
MINIO_ROOT_USER=minioadmin
MINIO_ROOT_PASSWORD=changeme-set-strong-password
MINIO_VOLUMES=""
MINIO_OPTS="--address :9000 --console-address :9001"
EOF
sudo chown root:minio /etc/default/minio
sudo chmod 640 /etc/default/minio
```

Shell

```
# Systemd unit
sudo tee /etc/systemd/system/minio.service >/dev/null <<'EOF'
[Unit]
```

```

Description=MinIO
Documentation=https://min.io/docs/minio/linux/index.html
Wants=network-online.target
After=network-online.target
AssertFileIsExecutable=/usr/local/bin/minio

[Service]
WorkingDirectory=/usr/local
User=minio
Group=minio
ProtectProc=invisible
EnvironmentFile=/etc/default/minio
ExecStartPre=/bin/bash -c "if [ -z \"${MINIO_VOLUMES}\" ]; then echo
\"MINIO_VOLUMES is not set\"; exit 1; fi"
ExecStart=/usr/local/bin/minio server $MINIO_OPTS $MINIO_VOLUMES
Restart=always
LimitNOFILE=1048576
TimeoutStopSec=infinity
SendSIGKILL=no

[Install]
WantedBy=multi-user.target
EOF

```

```

Shell
sudo systemctl daemon-reload
sudo systemctl enable minio

```

```

Shell
echo "=== Bootstrap complete on $(hostname) ==="
/usr/local/bin/minio --version
df -h /mnt/disk*

```

Load Balancer bootstrap script

Run this on every LB instance:

```

Shell
#!/usr/bin/env bash
set -euo pipefail

```

```

echo "=== Bootstrapping LB node $(hostname) ==="

sudo apt update
sudo apt upgrade -y
sudo apt install -y haproxy keepalived curl jq chrony
sudo systemctl enable --now chrony

# Stop and disable - Phase 6 configures and enables
sudo systemctl stop haproxy 2>/dev/null true
sudo systemctl disable haproxy 2>/dev/null true
sudo systemctl stop keepalived 2>/dev/null true
sudo systemctl disable keepalived 2>/dev/null true

echo "=== LB bootstrap complete on $(hostname) ==="
haproxy -v
keepalived -v

```

Push and run the script

For each node, scp the script up and execute over SSH:

```

Shell
# MinIO nodes (run in parallel)
for n in 1 2 3 4; do
    ip=$(oxide instance external-ip list --project minio-poc --instance
minio-inst-$n \
        jq -r '.items[] select(.kind=="ephemeral") .ip')
    scp -o StrictHostKeyChecking=accept-new bootstrap-minio.sh ubuntu@$ip:/tmp/
    ssh ubuntu@$ip "sudo bash /tmp/bootstrap-minio.sh" &
done
wait

# LB nodes - lb-1 reachable via Floating IP, lb-2 via ProxyJump through any MinIO
node
FIP=<floating-ip>
INST1_IP=<minio-inst-1 external IP>
LB2_PRIV=$(oxide instance nic list --project minio-poc --instance lb-2 \
    jq -r '.[] .ip_stack.value.v4.ip')

scp -o StrictHostKeyChecking=accept-new bootstrap-lb.sh ubuntu@$FIP:/tmp/
ssh ubuntu@$FIP "sudo bash /tmp/bootstrap-lb.sh"

scp -o StrictHostKeyChecking=accept-new -o ProxyJump=ubuntu@$INST1_IP \

```

```
bootstrap-lb.sh ubuntu@$LB2_PRIV:/tmp/
ssh -o ProxyJump=ubuntu@$INST1_IP ubuntu@$LB2_PRIV "sudo bash
/tmp/bootstrap-lb.sh"
```

Verify setup with below commands:

1. All four MinIO instances have `/mnt/disk1` through `/mnt/disk4` mounted with XFS, owned by `minio:minio`, with `/mnt/diskN/minio` subdirectory created.
2. All four MinIO instances have `/usr/local/bin/minio` installed, `minio.service` loaded and enabled (inactive/dead is correct - we haven't formed the cluster yet).
3. Both LB instances have HAProxy and keepalived installed (both services stopped and disabled).
4. `chronyc tracking` on every node shows healthy time sync.

Phase 5: MinIO install and cluster formation

Goal: distributed MinIO cluster running across all 4 nodes with EC:4 erasure coding, validated end-to-end via MinIO client, `mc`.

Name resolution

MinIO's cluster URL `http://minio-inst-{1...4}/mnt/disk{1...4}/minio` requires every node to resolve `minio-inst-1` through `minio-inst-4` by name. `/etc/hosts` is the simplest, predictable path.

Script (run from your laptop, picks up current private IPs from the API):

```
Shell
#!/usr/bin/env bash
set -euo pipefail

# Look up private IPs
NODE1_IP=$(oxide instance nic list --project minio-poc --instance minio-inst-1
jq -r '.[].ip_stack.value.v4.ip')
NODE2_IP=$(oxide instance nic list --project minio-poc --instance minio-inst-2
jq -r '.[].ip_stack.value.v4.ip')
NODE3_IP=$(oxide instance nic list --project minio-poc --instance minio-inst-3
jq -r '.[].ip_stack.value.v4.ip')
NODE4_IP=$(oxide instance nic list --project minio-poc --instance minio-inst-4
jq -r '.[].ip_stack.value.v4.ip')

cat > /tmp/minio-hosts-block <<EOF
# === MinIO cluster nodes ===
```

```

$NODE1_IP minio-inst-1
$NODE2_IP minio-inst-2
$NODE3_IP minio-inst-3
$NODE4_IP minio-inst-4
# === end MinIO cluster nodes ===
EOF

cat > /tmp/update-hosts.sh <<'OUTER'
#!/usr/bin/env bash
set -euo pipefail
sudo sed -i '/=== MinIO cluster nodes/,/=== end MinIO cluster nodes/d' /etc/hosts
sudo tee -a /etc/hosts < /tmp/minio-hosts-block > /dev/null
OUTER

for n in 1 2 3 4; do
    ext_ip=$(oxide instance external-ip list --project minio-poc --instance
minio-inst-$n \
        jq -r '.items[] select(.kind=="ephemeral") .ip')
    scp -o StrictHostKeyChecking=accept-new /tmp/minio-hosts-block
/tmp/update-hosts.sh ubuntu@$ext_ip:/tmp/
    ssh ubuntu@$ext_ip "bash /tmp/update-hosts.sh"
done

# Verify
for n in 1 2 3 4; do
    ext_ip=$(oxide instance external-ip list --project minio-poc --instance
minio-inst-$n \
        jq -r '.items[] select(.kind=="ephemeral") .ip')
    ssh ubuntu@$ext_ip 'for peer in 1 2 3 4; do getent hosts minio-inst-$peer;
done'
done

```

Each node should resolve all 4 peers to their private IPs.

Generate strong root credentials

Default *minioadmin/minioadmin* and any placeholder password must be replaced. Generate cryptographically random values, store immediately in a password manager.

```

Shell
ROOT_USER="minio-admin-$(openssl rand -hex 3)"
ROOT_PASSWORD=$(openssl rand -base64 32 tr -d '/+=' head -c 32)
echo "MINIO_ROOT_USER: $ROOT_USER"

```

```
echo "MINIO_ROOT_PASSWORD: $ROOT_PASSWORD"
```

Save to `1Password` or equivalent before proceeding.

Push the finalized `/etc/default/minio` to all 4 nodes

```
Shell
cat > /tmp/minio-env-file <<EOF
MINIO_ROOT_USER=$ROOT_USER
MINIO_ROOT_PASSWORD=$ROOT_PASSWORD
MINIO_VOLUMES="http://minio-inst-{1...4}/mnt/disk{1...4}/minio"
MINIO_OPTS="--address :9000 --console-address :9001"
EOF

for n in 1 2 3 4; do
    ext_ip=$(oxide instance external-ip list --project minio-poc --instance
minio-inst-$n \
        jq -r '.items[] select(.kind=="ephemeral") .ip')
    scp -o StrictHostKeyChecking=accept-new /tmp/minio-env-file
ubuntu@$ext_ip:/tmp/
    ssh ubuntu@$ext_ip "sudo install -o root -g minio -m 640 /tmp/minio-env-file
/etc/default/minio && rm /tmp/minio-env-file"
done
rm /tmp/minio-env-file
```

💡 `MINIO_VOLUMES` uses MinIO's three-dot brace expansion (`{1...4}`), not bash's two-dot. Bash won't expand it, MinIO will - the brace expression expands to 16 endpoints.

File ownership is `root:minio` with mode 640, so reading it back requires `sudo`.

Start the cluster

Start all four nodes in parallel so they discover each other within the formation window:

```
Shell
for n in 1 2 3 4; do
    ext_ip=$(oxide instance external-ip list --project minio-poc --instance
minio-inst-$n \
        jq -r '.items[] select(.kind=="ephemeral") .ip')
    ssh ubuntu@$ext_ip "sudo systemctl start minio" &
```

```
done
wait
sleep 15

# Health check
for n in 1 2 3 4; do
    ext_ip=$(oxide instance external-ip list --project minio-poc --instance
minio-inst-$n \
        jq -r '.items[] | select(.kind=="ephemeral") | .ip')
    result=$(ssh ubuntu@$ext_ip "curl -s -o /dev/null -w '%{http_code}'
http://localhost:9000/minio/health/live")
    echo "minio-inst-$n health: HTTP $result"
done
```

All four should report HTTP 200. The journalctl on each node should show *Formatting 1st pool, 1 set(s), 16 drives per set* confirming the EC:4 layout was initialized.

Validate with *mc*

External access to port 9000 is NOT open in the firewall (by design - production traffic goes through the LB on 443). For testing, use an SSH tunnel.

Install *mc* on your laptop:

```
Shell
brew install minio/stable/mc
```

Open a tunnel to inst-1:9000:

```
Shell
ssh -L 9000:localhost:9000 ubuntu@<inst-1-external-ip>
```

In another terminal, configure the alias against the tunnel:

```
Shell
export MINIO_PASSWORD="<from-password-manager>"
mc alias set minio-poc http://localhost:9000 <root-user> "$MINIO_PASSWORD"
unset MINIO_PASSWORD
```

```
mc admin info minio-poc
```

mc admin info should report:

- All 4 nodes online
- 16/16 drives online
- EC:4 listed in the erasure coding row
- Pool total capacity , 1.2 TiB usable (1.6 TiB raw with 75% efficiency)

Smoke test

```
Shell
mc mb minio-poc/test-bucket
echo "hello $(date)" > /tmp/smoke.txt
mc cp /tmp/smoke.txt minio-poc/test-bucket/
mc cat minio-poc/test-bucket/smoke.txt
mc rm minio-poc/test-bucket/smoke.txt
mc rb minio-poc/test-bucket
```

If you see the file written and read back, end-to-end S3 is functional.

Verify setup with below commands:

1. All 4 MinIO nodes report HTTP 200 on the health endpoint.
2. *mc admin info* shows 16/16 drives online with EC:4.
3. A PUT/GET roundtrip against the cluster succeeds.
4. The placeholder *changeme-set-strong-password* is no longer in any */etc/default/minio*.

Phase 6: HAProxy + watcher + FIP failover

Goal: production-style S3 endpoint via the Floating IP, TLS terminated at HAProxy, round-robin across all 4 MinIO backends, with autonomous failover from lb-1 to lb-2 if the primary LB dies.

Hosts entries on LBs

Both LBs need to resolve *minio-inst-1* through *minio-inst-4* so HAProxy can reach backends by name. Same */etc/hosts* block script as Phase 5, pushed to lb-1 (via FIP) and lb-2 (via ProxyJump through a MinIO node).

Self-signed TLS certificate

POC-grade. Real PKI (Let's Encrypt or customer CA) replaces this in hardening.

```
Shell
openssl genrsa -out minio-lb.key 2048
openssl req -new -x509 -key minio-lb.key -out minio-lb.crt -days 365 \
  -subj "/CN=minio-s3-endpoint" \
  -addext "subjectAltName=IP:<FLOATING_IP>,DNS:s3.minio-poc.local"
cat minio-lb.crt minio-lb.key > minio-lb.pem
chmod 600 minio-lb.pem
```

Push to `/etc/haproxy/certs/minio-lb.pem` on both LBs, mode 600 owned by root.

HAProxy config

```
None
global
  daemon
  maxconn 4096
  log /dev/log local0
  log /dev/log local1 notice
  ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets
  ssl-default-bind-ciphers
  ECDHE+AESGCM:ECDHE+CHACHA20:DHE+AESGCM:DHE+CHACHA20:!aNULL:!MD5
  tune.ssl.default-dh-param 2048

defaults
  log global
  mode http
  option httplog
  option dontlognull
  option forwardfor
  option http-server-close
  timeout connect 10s
  timeout client 5m
  timeout server 5m
  timeout http-request 30s
  timeout http-keep-alive 30s
  timeout queue 1m
  timeout tunnel 10m

listen stats
  bind 127.0.0.1:8404
```

```

stats enable
stats uri /
stats refresh 5s

frontend minio_https
  bind :443 ssl crt /etc/haproxy/certs/minio-lb.pem alpn http/1.1
  default_backend minio_backend

backend minio_backend
  balance roundrobin
  option httpchk
  http-check send meth GET uri /minio/health/live ver HTTP/1.1 hdr Host
minio.local
  http-check expect status 200
  default-server check inter 5s rise 2 fall 3 maxconn 1000
  server minio-inst-1 minio-inst-1:9000
  server minio-inst-2 minio-inst-2:9000
  server minio-inst-3 minio-inst-3:9000
  server minio-inst-4 minio-inst-4:9000

```

Push, validate with `haproxy -c -f /etc/haproxy/haproxy.cfg`, enable + start.

Smoke test via FIP

Point *mc* at the Floating IP directly (no SSH tunnel - the LB is the production endpoint):

```

Shell
mc alias set --insecure minio-poc https://<FLOATING_IP> <root-user>
"$MINIO_PASSWORD"
mc --insecure admin info minio-poc

```

Should return all 4 nodes, 16 drives, EC:4. PUT/GET via FIP should succeed.

Failover mechanism (oxide CLI + credentials + watcher)

The failover trigger lives on the LBs themselves so it works without a human in the loop.

Install Oxide CLI on both LBs

Download the matching Linux binary from the oxide.rs GitHub release (asset name `oxide-cli-x86_64-unknown-linux-gnu.tar.xz`), extract, install to `/usr/local/bin/oxide`.

Copy laptop's credentials to root on each LB

The CLI needs auth to call the floating-IP API. Copy `~/.config/oxide/credentials.toml` and `config.toml` from your laptop to `/root/.config/oxide/` on each LB, mode 600 owned by root.

This grants the LB full silo access. For production, replace with a scoped service account that has only `floating-ip:attach/detach` permissions.

Failover script (`/usr/local/bin/minio-fip-failover.sh`) - idempotent. Reads the current FIP holder, compares to local instance UUID, detaches and reattaches if needed, no-ops if FIP is already local:

```
Shell
#!/usr/bin/env bash
set -euo pipefail
OXIDE_HOST="https://<your-silo>.sys.<rack>.oxide-preview.com"
PROJECT="minio-poc"
FIP_NAME="minio-s3-endpoint"
PROFILE="<your-profile-name>"
THIS_INSTANCE="$(hostname)"
export OXIDE_HOST

logger -t minio-failover "Starting; target = $THIS_INSTANCE"
CURRENT=$(oxide floating-ip view --project "$PROJECT" --floating-ip "$FIP_NAME"
--profile "$PROFILE" jq -r '.instance_id // empty')
THIS_INSTANCE_ID=$(oxide instance view --project "$PROJECT" --instance
"$THIS_INSTANCE" --profile "$PROFILE" jq -r '.id')

if [ "$CURRENT" = "$THIS_INSTANCE_ID" ]; then
    logger -t minio-failover "FIP already on $THIS_INSTANCE, no-op"
    exit 0
fi

[ -n "$CURRENT" ] && oxide floating-ip detach --project "$PROJECT" --floating-ip
"$FIP_NAME" --profile "$PROFILE"
oxide floating-ip attach --project "$PROJECT" --floating-ip "$FIP_NAME" --kind
instance --parent "$THIS_INSTANCE" --profile "$PROFILE"
logger -t minio-failover "Failover complete"
```

Watcher script (`/usr/local/bin/minio-lb-watcher.sh`) - polls peer's TCP 443, fires failover on 3 consecutive failures:

```
Shell
#!/usr/bin/env bash
```

```

set -uo pipefail
PEER_HOST="${1:?Usage: $0 <peer-ip>}"
PEER_PORT="${2:-443}"
CHECK_INTERVAL="${3:-2}"
FAIL_THRESHOLD="${4:-3}"

logger -t minio-lb-watcher "Starting; peer=$PEER_HOST:$PEER_PORT"
failures=0
last_state="init"
while true; do
    if nc -z -w2 "$PEER_HOST" "$PEER_PORT" 2>/dev/null; then
        [ "$last_state" != "up" ] && logger -t minio-lb-watcher "Peer $PEER_HOST UP"
        last_state="up"; failures=0
    else
        failures=$((failures + 1))
        logger -t minio-lb-watcher "Peer check FAILED ($failures/$FAIL_THRESHOLD)"
        if [ "$failures" -ge "$FAIL_THRESHOLD" ] && [ "$last_state" != "down" ]; then
            logger -t minio-lb-watcher "Peer DOWN, invoking failover"
            /usr/local/bin/minio-fip-failover.sh logger -t minio-lb-watcher "Failover
non-zero"
            last_state="down"
        fi
    fi
    sleep "$CHECK_INTERVAL"
done

```

Systemd unit (*/etc/systemd/system/minio-lb-watcher.service*) - one per LB, each pointing at the OTHER LB's private IP:

```

Shell
[Unit]
Description=MinIO LB watcher (TCP-polling FIP failover)
After=network-online.target haproxy.service
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/local/bin/minio-lb-watcher.sh <peer-private-ip> 443 2 3
Restart=always
RestartSec=5
User=root

[Install]

```

```
WantedBy=multi-user.target
```

Execute below command on both lbs:

```
Shell
systemctl enable --now minio-lb-watcher
```

Failover drill

Kill HAProxy on lb-1, watch FIP move, confirm mc still works:

```
Shell
ssh lb-1 "sudo systemctl stop haproxy"
sleep 15
oxide floating-ip view --project minio-poc --floating-ip minio-s3-endpoint jq -r
'.instance_id' # should be lb-2's UUID
mc --insecure admin info minio-poc # should return cluster view via lb-2
ssh lb-1 "sudo systemctl start haproxy"
```

💡 Measured failover time: ~11 seconds end to end. 6 seconds for the watcher to mark peer DOWN (3 x 2s checks), ~4 seconds for Oxide API detach + attach round trip.

Verify setup with below commands:

1. HAProxy active on both LBs, all 4 MinIO backends UP in HAProxy's view
2. *mc --insecure admin info* works via the Floating IP
3. PUT/GET via FIP succeeds
4. Failover drill: kill HAProxy on lb-1, FIP moves to lb-2 within ~15 seconds, mc continues working transparently
5. After restoring lb-1's HAProxy, FIP stays sticky on lb-2 (no auto-bounce)

Architecture FAQ

The "why" behind every design call in this build. Read this before changing any of the choices below.

Software choices

Why MinIO and not Ceph, Swift, or RustFS?

MinIO is production-proven at petabyte scale, ships a single Go binary, and has the deepest S3-API fidelity outside AWS itself. Ceph is operationally heavier and not S3-native. Swift's S3 compatibility layer lags behind. RustFS is interesting but pre-GA as of May 2026 with no named production customers and unstable cluster mode. We pick the boring, proven choice.

Why HAProxy instead of NGINX?

Both work. HAProxy has cleaner TCP load balancing for S3 backend health checks, lower memory footprint, and is the more common choice in production object-storage setups. If a customer has NGINX expertise in-house, swap freely. The architecture doesn't care.

Why customer-built LB instead of an Oxide-native one?

Oxide does not ship a managed load balancer in the current product. RFD 21 explicitly defers it. The documented workaround is Floating IP + a guest VM running HAProxy. We accept the gap and build the LB tier ourselves. When Oxide ships a managed LB, this pattern can collapse to that.

Why systemd, not containers or Kubernetes?

A 4-node MinIO cluster does not need an orchestrator. systemd-managed binaries are simpler to install, simpler to upgrade, simpler to debug. Kubernetes adds operational surface area without benefit for this workload. The MinIO binary is mature enough that the OS-level service is the right abstraction.

Why Ubuntu 24.04 LTS?

Best MinIO documentation coverage, current LTS through 2029, default for cloud workloads. Debian and Rocky also work. Easy to swap if customer requires a specific distro.

Topology

Why 4 MinIO instances?

Minimum viable distributed MinIO cluster that survives a full instance failure. With 4 drives per instance, the cluster has 16 drives total, which gives clean EC:4 math. Going to 6 or 8 nodes increases failure tolerance and parallelism but doubles the operational footprint. 4 is the sweet spot for a reference build.

Why 4 drives per instance?

16 total drives is the smallest multiple of 4 that supports EC:4 (12 data + 4 parity). 4 drives per instance also means one full instance loss equals exactly 4 drive failures, which sits right at the EC parity limit. The cluster recovers cleanly from one instance loss but no more.

Why 2 LB instances?

One LB is a single point of failure. Two LBs in active/standby with Floating IP failover means an LB process or VM dying doesn't take down the S3 endpoint. We can't run them active/active because Oxide Floating IPs attach to exactly one instance at a time today.

Why a dedicated VPC (`minio-vpc`) instead of the default VPC?

Naming clarity (a VPC literally named `default` tells you nothing six months from now), blast-radius isolation, and template discipline. Production customer deployments will reuse the project for other workloads, and a dedicated VPC per service is the right pattern.

In a distributed MinIO cluster, which node "owns" a bucket or object?

None of them. Any node can answer any S3 request. The node a client connects to becomes the temporary coordinator for that request; it reads or writes shards across the cluster via internal traffic and returns the result. Bucket metadata lives in a cluster-wide hidden `.minio.sys/` directory that's replicated across all nodes, so the bucket is visible from every node simultaneously. Object data is split into 16 erasure-coded shards (12 data + 4 parity) and spread across the 16 drives on the 4 nodes; a single 1 MiB object physically touches all 4 instances. This is why the LB can round-robin client traffic without any node-affinity logic, and why losing one node doesn't lose any data.

Storage choices

Why local disks for data, but distributed for the boot disk?

Two backends for two reasons. Data disks are local because MinIO's erasure coding already handles redundancy at the application layer. Stacking that on top of distributed Crucible disks (3x replicated) would double-replicate the data for ~4x raw overhead. Local keeps overhead at 1.33x. Oxide's docs explicitly name "distributed databases

with application-level replication" as the local-disk use case. Boot disks are distributed because Oxide's local disks today are hardcoded to block size 4096, and the Ubuntu cloud image has GPT laid out for 512-byte sectors. A local boot disk from that image won't boot (UEFI shell). Distributed boot disks respect the requested block size. Cost: 3x replication on 30 GiB boot disks, small in absolute terms.

Why EC:4 (12 data + 4 parity) instead of mirroring or other parity counts?

75% storage efficiency with tolerance for 4 simultaneous drive failures. Mirroring (3x copies) is 33% efficient, too wasteful at scale. Single-parity (RAID-5 equivalent) tolerates only 1 failure, too fragile. EC:4 sits at the industry sweet spot for object storage and matches our "tolerate one full instance loss" guarantee.

Why 100 GiB data disks and 30 GiB boot disks?

100 GiB per data disk gives 1.6 TiB raw / 1.2 TiB usable, plenty for POC validation while staying under typical silo quota. 30 GiB boot is comfortable for Ubuntu plus updates plus logs plus future tooling, without bloating. Scale both up for production workloads.

Why block size 4096 on data disks but 512 on the boot disk?

4096 aligns with modern SSD physical block size; better I/O efficiency for the actual workload. The Ubuntu cloud image has its GPT partition table laid out for 512-byte sectors. Override to 4096 on the boot image and the OS fails to find the partition table where it expects. Match the image's block size on boot disks, pick 4096 on data disks where the layout is yours.

Why is sled placement verification mandatory?

Local disks pin instances to their sled. If two MinIO instances co-locate, that sled is a 2-node failure domain and EC:4 cannot tolerate its loss. Oxide does NOT guarantee instance spread by default - the scheduler is free to co-locate. The right fix is an anti-affinity group with *policy: fail* and *failure_domain: sled*. The scheduler then refuses to co-locate and errors loudly if it can't satisfy the constraint. Verify placement after every instance create with *sled-map.sh*.

Why does the bootstrap script discover the boot disk dynamically instead of assuming *nvme0n1*?

Because Oxide doesn't guarantee NVMe device naming order. On three of the four MinIO instances built here, the boot disk landed at *nvme0n1* and data disks at *nvme1n1* through *nvme4n1*. On the fourth instance, the boot disk landed at *nvme3n1* with data disks at *nvme0n1*, *nvme1n1*, *nvme2n1*, and *nvme4n1*. A naive script that runs *mkfs.xfs /dev/nvme0n1* would format the wrong disk on the fourth instance. The bootstrap uses *findmnt -no SOURCE /* to discover where root is mounted, then formats every other NVMe device.

Networking choices

Why a Floating IP and not DNS round-robin?

DNS round-robin has no health awareness. A dead MinIO node still gets clients until DNS caches expire, which can take minutes. A Floating IP routed through HAProxy gives a single stable endpoint with sub-second health-driven failover at the LB layer. DNS-RR is a fine fallback for non-tier-1 workloads but not for a primary object store.

Why TLS terminated at the LB rather than end-to-end to MinIO?

Single certificate management point. LB-to-MinIO traffic stays inside the VPC on private addresses where Oxide's underlay encrypts in transit. If a customer compliance regime requires end-to-end (very few do), we can re-encrypt at the LB and present certificates to MinIO too. Default is LB-terminated.

Why ephemeral external IPs during the build, but planning to detach later?

SSH access for setup is easiest with a direct external IP per node. After the cluster is configured, the only external entry point should be the LB's Floating IP. Detaching the per-instance external IPs reduces attack surface and gives IP-pool addresses back to the silo. The hardening pass strips them.

Why doesn't the firewall allow port 9000 from the internet, even though clients need to reach MinIO?

Because clients shouldn't reach MinIO directly. Exposing 9000 externally would skip the LB layer (no health-aware failover, no TLS termination, plaintext credentials over the internet) and create extra attack surface for no benefit. For ad-hoc testing during the build, use an SSH tunnel: `ssh -L 9000:localhost:9000 ubuntu@<minio-node>`. The tunnel only exists while you have the SSH session open, no persistent firewall hole.

Why is the Floating IP sticky after failover instead of bouncing back to the primary?

Because preempting on every primary-recovery creates flapping. If lb-1's HAProxy restarts cleanly but lb-2 had taken the FIP during a brief blip, an aggressive preempt-back means the FIP moves twice (lb-1 → lb-2 → lb-1), with both moves causing ~10 seconds of client disruption. Sticky behavior moves the FIP once, then leaves it until that holder fails. Same total outage during a real failure, no second outage during recovery. To intentionally fail back to lb-1 (e.g., during maintenance windows), run `oxide floating-ip detach && oxide floating-ip attach --parent lb-1` manually.